UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Transparência Verificável em Modelos de IA por Meio da Auditoria de Datasets

Cristian Soares de Souza Chagas Filho

Orientador: Prof. Dr. Eduardo Lucena Falcão

Natal, RN, 8 de dezembro de 2025

UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
CENTRO DE TECNOLOGIA
GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

Transparência Verificável em Modelos de IA por Meio da Auditoria de Datasets

Cristian Soares de Souza Chagas Filho

Orientador: Prof. Dr. Eduardo Lucena Falcão

Trabalho de Conclusão de Curso de Graduação na modalidade Monografia, submetido como parte dos requisitos necessários para conclusão do curso de Engenharia de Computação pela Universidade Federal do Rio Grande do Norte (UFRN/CT).

Natal, RN, 8 de dezembro de 2025

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Chagas Filho, Cristian Soares de Souza.
Transparência verificável em modelos de IA por meio da auditoria de datasets / Cristian Soares de Souza Chagas Filho. - 2025.
46 f.: il.

Monografia (graduação) - Universidade Federal do Rio Grande do Norte, Centro de Tecnologia, Curso de Engenharia de Computação, Natal, RN, 2025.
Orientação: Prof. Dr. Eduardo Lucena Falcão.

1. Integridade de Dados - Monografia. 2. Árvores de Merkle - Monografia. 3. Auditoria de IA - Monografia. I. Falcão, Eduardo Lucena. II. Título.

RN/UF/BCZM CDU 004.056.2

Elaborado por Fernanda de Medeiros Ferreira Aquino - CRB-15/301

Agradecimentos

Ao meu orientador, Prof. Dr. Eduardo Lucena Falcão, que me introduziu ao tema deste trabalho e me guiou de forma direta e objetiva, fundamental para alcançarmos os melhores resultados.

Aos meus pais, que sempre me incentivaram a estudar e me guiaram nas encruzilhadas da vida, mas que, acima de tudo, me deram liberdade para tomar as minhas próprias decisões.

À minha irmã, minha eterna inspiração profissional. Sou muito feliz por compartilhar a mesma formação que você e poder dizer que, agora, somos irmãos também de profissão.

À minha namorada, que está comigo diariamente oferecendo apoio incondicional. Obrigado por me incentivar a ser melhor e por me mostrar que todas as lutas que enfrentamos valem a pena.

Aos meus amigos, que tornaram esta trajetória mais leve e divertida, enfrentando ao meu lado todos os desafios que o curso impôs.

Resumo

A crescente utilização de modelos de Inteligência Artificial (IA) em aplicações críticas tem evidenciado a necessidade de mecanismos robustos para garantir a transparência e a integridade dos dados utilizados em seu treinamento. Atualmente, a ausência de verificabilidade na cadeia de suprimentos de aprendizado de máquina expõe modelos a riscos de segurança, como envenenamento de dados (*data poisoning*), e dificulta a auditoria externa. Este trabalho apresenta o desenvolvimento e a validação da ferramenta `hf-merkle`, uma biblioteca que integra estruturas criptográficas de Árvores de Merkle ao ecossistema Hugging Face Hub. A solução permite gerar provas de inclusão imutáveis para datasets, publicar essas provas automaticamente nos repositórios de modelos e realizar verificações independentes via interface de linha de comando (CLI). Os experimentos realizados demonstraram a viabilidade técnica da abordagem: embora o tempo de geração da árvore e o tamanho do arquivo de prova apresentem crescimento linear em relação ao número de arquivos, a latência para verificação de integridade mostrou-se extremamente baixa. A ferramenta contribui, assim, para a governança algorítmica, fornecendo um meio prático e descentralizado para assegurar a proveniência dos dados em pipelines de IA.

Palavras-chave: Integridade de Dados. Árvores de Merkle. Hugging Face. Auditoria de IA. Cadeia de Suprimentos de ML.

Abstract

The increasing use of Artificial Intelligence (AI) models in critical applications has highlighted the need for robust mechanisms to ensure the transparency and integrity of the data used in their training. Currently, the lack of verifiability in the machine learning supply chain exposes models to security risks, such as data poisoning, and hinders external auditing. This work presents the development and validation of the `hf-merkle` tool, a library that integrates Merkle Tree cryptographic structures into the Hugging Face Hub ecosystem. The solution allows for the generation of immutable inclusion proofs for datasets, the automatic publication of these proofs in model repositories, and independent verification via a Command Line Interface (CLI). The experiments conducted demonstrated the technical feasibility of the approach: although the tree generation time and proof file size exhibit linear growth relative to the number of files, the latency for integrity verification proved to be extremely low. Thus, the tool contributes to algorithmic governance by providing a practical and decentralized means to ensure data provenance in AI pipelines.

Keywords: Data Integrity. Merkle Trees. Hugging Face. AI Auditing. ML Supply Chain.

Sumário

Sumário i

Lista de Figuras iii

Lista de Tabelas v

1 Introdução 1
- 1.1 Motivação . . . 1
- 1.2 Objetivos . . . 2
- 1.3 Contribuições . . . 2
- 1.4 Estrutura do Trabalho . . . 2

2 Fundamentação Teórica 5
- 2.1 Modelos de Inteligência Artificial . . . 5
- 2.2 Processo de Treinamento e a Opacidade de Dados . . . 5
- 2.3 Árvores de Merkle . . . 6
  - 2.3.1 Estrutura e Construção . . . 6
- 2.4 Verificação e Provas de Inclusão . . . 7
  - 2.4.1 Exemplo de Verificação . . . 7

3 Revisão de Literatura 9
- 3.1 Trabalhos Relacionados . . . 9
- 3.2 Metodologia de pesquisa . . . 10
- 3.3 Artigos relevantes ao tema . . . 11
  - 3.3.1 Transparência e verificabilidade em modelos de IA . . . 11
  - 3.3.2 Segurança da cadeia de suprimentos de Machine Learning . . . 11
  - 3.3.3 Verificação criptográfica e provas de inclusão . . . 11
  - 3.3.4 Repositórios de modelos e desafios de segurança . . . 11
- 3.4 Produção científica sobre o tema . . . 12
- 3.5 Metanálise . . . 12

4 Metodologia 13
- 4.1 Tipo e natureza da pesquisa . . . 13
- 4.2 Local e ambiente do estudo . . . 13
- 4.3 Materiais e ferramentas utilizadas . . . 13
- 4.4 Procedimentos metodológicos (etapas da pesquisa) . . . 14

4.4.1 Revisão bibliográfica e levantamento de requisitos 14
4.4.2 Projeto da arquitetura 14
4.4.3 Implementação 14
4.4.4 Empacotamento e interface 15
4.4.5 Testes e validação 15
4.4.6 Medições de desempenho 15
4.4.7 Análise qualitativa e documentação 15
4.4.8 Reprodutibilidade e recomendações 15
4.5 Instrumentos de coleta e métricas 16
4.6 Considerações éticas 16

5 Implementação **19**
5.1 Algoritmos Implementados 19
5.1.1 Cálculo do hash dos arquivos 19
5.1.2 Construção da Merkle Tree 19
5.1.3 Geração da Merkle Proof 20
5.1.4 Verificação da prova 21
5.2 Integração com Hugging Face Hub 22
5.3 Automação via CLI 22
5.4 Síntese da Implementação 22

6 Experimentos e Resultados **23**
6.1 Objetivo dos experimentos 23
6.2 Configuração experimental 23
6.3 Metodologia de medição e agregação 24
6.4 Apresentação dos resultados 24
6.4.1 Tempo de geração vs número de folhas 24
6.4.2 Tamanho do arquivo de prova 25
6.4.3 Throughput de hashing 25
6.4.4 Latência de verificação — positivas vs negativas 25
6.4.5 Efeito da duplicação 26

7 Conclusão **27**

Referências bibliográficas **29**

Lista de Figuras

2.1 Estrutura básica de uma Árvore de Merkle. 7

5.1 Fluxo percorrido para gerar a Merkle Tree do dataset 21
5.2 Fluxo percorrido para verificação da Merkle Root 21

6.1 Tempo de geração da Merkle Tree em função do número de folhas (escala log-log). Cada ponto representa a média de várias repetições para um dataset com número N de arquivos. 24
6.2 Tamanho do arquivo de prova (em KB) em função do número de folhas. Observa-se crescimento aproximadamente linear com N. 25
6.3 Taxa efetiva de hashing (MB/s) em função do tamanho total do dataset (MB). 26
6.4 Distribuição das latências de verificação (positiva, à esquerda; negativa, à direita). 26

Lista de Tabelas

4.1 Etapas cronológicas da metodologia para desenvolvimento da ferramenta hf-merkle 17

Capítulo 1

Introdução

Nos últimos anos, o crescente uso de modelos de Inteligência Artificial (IA) em aplicações críticas, como diagnósticos médicos, sistemas de recomendação, análise forense, reconhecimento biométrico e tomada de decisão automatizada, intensificou a preocupação com a transparência, a confiabilidade e a integridade dos dados utilizados no processo de treinamento. A qualidade e a origem dos dados são fatores determinantes para o comportamento, a precisão e os potenciais vieses presentes em sistemas baseados em aprendizado de máquina. Entretanto, mesmo em ambientes de pesquisa e desenvolvimento profissional, a garantia de que o conjunto de dados divulgado como fonte de treinamento corresponde fielmente ao utilizado na prática ainda representa um desafio técnico e processual significativo.

1.1 Motivação

A opacidade referente aos dados de treinamento é um dos problemas centrais na governança atual da Inteligência Artificial. Atualmente, não existe um mecanismo padronizado e amplamente adotado que permita a terceiros verificar, de forma criptográfica e irrefutável, quais dados compuseram o *dataset* de treinamento de um modelo específico. Essa lacuna abre margem para sérios problemas éticos e legais.

Muitos modelos de grande porte (LLMs e modelos de difusão) têm consumido volumes massivos de dados da internet de forma inadvertida ou obscura. Isso levanta questões críticas sobre o uso de material protegido por direitos autorais, a apropriação indevida de obras de artistas e criadores de conteúdo sem consentimento, e a inclusão de dados sensíveis ou enviesados que podem perpetuar discriminações. Sem uma prova de inclusão verificável, torna-se quase impossível para auditores ou detentores de direitos confirmarem se um determinado dado foi ou não utilizado no treinamento de uma IA.

Nesse cenário, emerge a necessidade de iniciativas voltadas à auditabilidade e à reprodutibilidade, aproximando a área de IA de práticas robustas de governança. Uma abordagem promissora envolve o uso de estruturas criptográficas, como as árvores de Merkle (*Merkle Trees*). Amplamente utilizadas em sistemas distribuídos e *blockchain*, essas estruturas permitem gerar uma representação compacta e única de um conjunto de dados, possibilitando comprovar a pertinência de uma informação ao conjunto original sem a necessidade de expor todo o volume de dados, mitigando problemas de privacidade e custos

de armazenamento.

1.2 Objetivos

O objetivo geral deste trabalho é explorar, implementar e avaliar a aplicação de estruturas Merkle como mecanismo de verificação de integridade e transparência em *pipelines* de treinamento de IA.

Para alcançar este propósito, foram definidos os seguintes objetivos específicos:

- Desenvolver uma ferramenta computacional (CLI) integrada ao ecossistema Hugging Face para automatizar a geração de provas de integridade;
- Implementar algoritmos de geração de Árvores de Merkle capazes de processar diretórios de dados e exportar a *Merkle Root* e provas de inclusão;
- Criar mecanismos de verificação que permitam a usuários externos confirmarem, de forma independente, se um arquivo específico fez parte do treinamento;
- Avaliar a viabilidade da solução proposta em termos de desempenho e facilidade de integração com fluxos de trabalho existentes.

1.3 Contribuições

A principal contribuição deste trabalho é o desenvolvimento e a disponibilização do **hf-merkle**, uma extensão para o ecossistema Hugging Face que permite gerar, publicar e verificar provas criptográficas de integridade de *datasets*.

Diferentemente de abordagens puramente teóricas, esta pesquisa entrega uma solução prática que inclui:

1. **Mecanismo Proposto:** Uma arquitetura de verificação baseada em Merkle Trees adaptada para o versionamento de dados em repositórios de modelos de IA.
2. **Implementação para o Framework HF:** A integração de uma interface de linha de comando com as APIs oficiais do Hugging Face Hub, facilitando a adoção pela comunidade de desenvolvedores.
3. **Validação do Mecanismo:** A demonstração da eficácia da ferramenta na geração de provas de inclusão e na detecção de adulterações nos conjuntos de dados.
4. **Avaliação de Desempenho:** A análise do custo computacional e temporal para a geração das árvores e provas em diferentes volumes de dados, demonstrando a escalabilidade da solução.

Essa solução busca apoiar iniciativas de governança algorítmica e conformidade regulatória, como o AI Act europeu, fornecendo um mecanismo simples e verificável para pesquisadores e empresas.

1.4 Estrutura do Trabalho

Este documento está organizado em sete capítulos, estruturados da seguinte forma:

O **Capítulo 2** apresenta a fundamentação teórica necessária para a compreensão do trabalho, abordando conceitos essenciais sobre integridade de dados, funções de hash, o funcionamento das Árvores de Merkle e o ecossistema Hugging Face.

O **Capítulo 3** traz a revisão da literatura, contextualizando o estado da arte em relação à proveniência de dados, auditoria em sistemas de IA e trabalhos correlatos que utilizam estruturas criptográficas para verificação de datasets.

O **Capítulo 4** descreve a metodologia adotada para o desenvolvimento da solução, detalhando a arquitetura do sistema, os requisitos levantados e o fluxo de dados proposto para a certificação de integridade.

O **Capítulo 5** detalha a implementação técnica da ferramenta *hf-merkle*, discutindo as decisões de projeto, as tecnologias utilizadas e a integração prática com as APIs do Hugging Face Hub.

O **Capítulo 6** apresenta os experimentos e resultados, demonstrando a validação funcional da ferramenta e a análise de desempenho quanto ao tempo de processamento e uso de recursos.

Por fim, o **Capítulo 7** apresenta as conclusões do estudo, sintetizando as contribuições alcançadas, as limitações identificadas e as sugestões para trabalhos futuros.

Capítulo 2

Fundamentação Teórica

Este capítulo apresenta os conceitos fundamentais que embasam este trabalho. Inicialmente, discute-se a definição de modelos de Inteligência Artificial e o funcionamento geral de seus processos de treinamento, destacando a lacuna existente quanto à transparência e rastreabilidade dos dados. Em seguida, são apresentados os conceitos de estruturas de dados criptográficas, com foco nas Árvores de Merkle, detalhando sua arquitetura e o algoritmo de prova de inclusão, mecanismos essenciais para a solução de auditoria proposta nesta pesquisa.

2.1 Modelos de Inteligência Artificial

Em sua definição mais ampla, um modelo de Inteligência Artificial (IA) é uma representação matemática de um processo do mundo real, construída a partir de algoritmos que identificam padrões em conjuntos de dados. Diferentemente da programação tradicional, onde regras lógicas são explicitamente codificadas por humanos, os modelos de aprendizado de máquina (*Machine Learning*) induzem essas regras a partir de exemplos fornecidos durante uma fase de treinamento.

Atualmente, as arquiteturas mais predominantes, especialmente no contexto de Processamento de Linguagem Natural (PLN) e Visão Computacional, baseiam-se em Redes Neurais Profundas (*Deep Learning*). Essas redes são compostas por camadas de neurônios artificiais, funções matemáticas ajustáveis, que transformam os dados de entrada (como uma imagem ou um texto) em uma saída desejada (uma classificação ou uma predição), através de operações matriciais não lineares.

A "inteligência"do modelo reside em seus parâmetros (pesos e viéses). Um modelo moderno, como os Grandes Modelos de Linguagem (LLMs), pode conter bilhões de parâmetros, tornando-se uma "caixa preta"onde é difícil interpretar como uma decisão específica foi tomada ou, crucialmente, qual dado específico influenciou aquela decisão.

2.2 Processo de Treinamento e a Opacidade de Dados

O treinamento de um modelo de IA é o processo computacional de ajuste iterativo de seus parâmetros para minimizar o erro em suas previsões. Em linhas gerais, este fluxo

ocorre da seguinte maneira:

1. **Coleta e Preparação de Dados:** Reúne-se um *dataset* (conjunto de dados), que passa por etapas de limpeza, formatação e tokenização.
2. **Forward Pass (Propagação):** O dado passa pela rede neural, gerando uma predição.
3. **Cálculo de Perda (Loss):** Compara-se a predição com o resultado real esperado, calculando-se o erro.
4. **Backpropagation (Retropropagação):** Utilizando algoritmos de otimização (como o Gradiente Descendente), o erro é propagado de volta pela rede, ajustando levemente os pesos para que o modelo acerte mais na próxima vez.

Apesar da sofisticação matemática deste processo, existe uma deficiência crítica na maioria dos *frameworks* modernos de IA (como PyTorch ou TensorFlow): a perda do vínculo com a origem.

Ao final do treinamento, o resultado é um arquivo binário contendo apenas os pesos numéricos (o modelo treinado). O vínculo direto com os dados brutos utilizados é rompido. Não há, na arquitetura padrão de treinamento, um "rastro"criptográfico que prove que o arquivo X foi utilizado para gerar o modelo Y.

Embora existam diferentes abordagens de aprendizado (supervisionado, não supervisionado, por reforço), nenhuma delas provê, nativamente, mecanismos de transparência e auditoria de dados. A integridade do processo depende inteiramente de registros manuais ou metadados textuais (como *Model Cards*), que são suscetíveis a erros humanos ou adulterações maliciosas. Essa ausência de garantia técnica é o problema central que motiva o uso de estruturas criptográficas externas.

2.3 Árvores de Merkle

Para endereçar o problema de integridade e verificação em grandes volumes de dados, utilizam-se estruturas criptográficas conhecidas como Árvores de Merkle (*Merkle Trees*). Propostas por Ralph Merkle em 1979, essas estruturas são árvores binárias de *hash* que permitem resumir um grande conjunto de dados em uma única assinatura digital fixa, denominada Raiz de Merkle (*Merkle Root*).

2.3.1 Estrutura e Construção

A construção de uma Árvore de Merkle ocorre de baixo para cima (bottom-up):

1. **Folhas (Leaves):** Cada bloco de dados (ou arquivo do dataset) é processado por uma função de *hash* criptográfica (como SHA-256), gerando os nós folha.
2. **Nós Intermediários:** Os *hashes* das folhas são agrupados em pares e concatenados. Um novo *hash* é gerado a partir dessa concatenação.
3. **Raiz (Root):** O processo se repete recursivamente até restar apenas um único *hash* no topo da árvore.

A Figura 2.1 ilustra essa composição. Se qualquer bit de um arquivo na base da árvore for alterado, o seu *hash* mudará. Consequentemente, o *hash* de seu nó pai também mudará, propagando a alteração até a raiz. Isso garante a propriedade de "resistência a adulteração"(*tamper-resistance*).

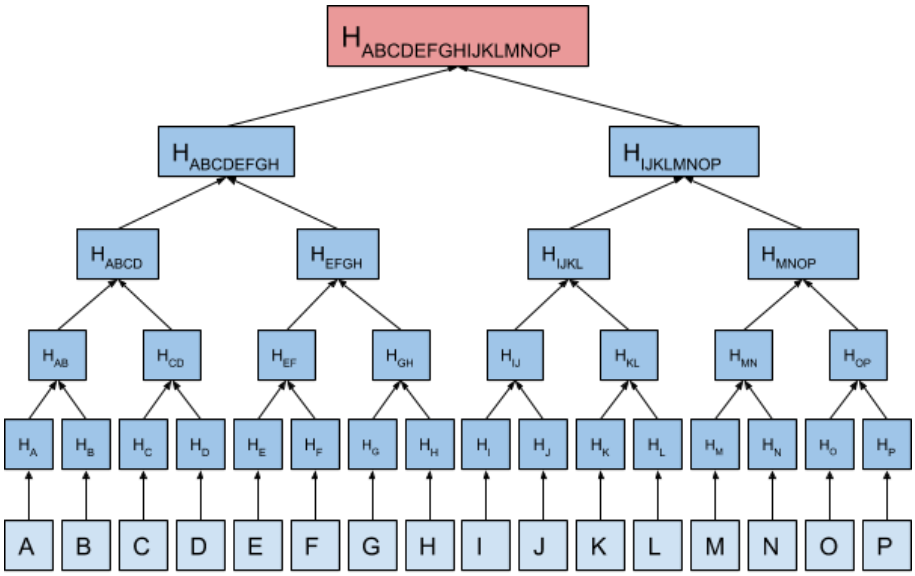

Figura 2.1: Estrutura básica de uma Árvore de Merkle.

2.4 Verificação e Provas de Inclusão

A principal utilidade da Árvore de Merkle em sistemas de auditoria não é apenas o armazenamento eficiente de hashes, mas a capacidade de provar que um determinado dado pertence ao conjunto original sem a necessidade de revelar ou acessar todo o conjunto de dados. Isso é chamado de Prova de Inclusão (*Merkle Proof*).

Para verificar se um arquivo de dados D (cujo hash é H_D) está contido na árvore definida pela Raiz R, não é necessário baixar o dataset inteiro. O verificador precisa apenas de:

1. O dado D (para calcular H_D); 2. A Raiz R (que deve ser pública e confiável); 3. O "Caminho de Auditoria"(*Audit Path*).

O caminho de auditoria consiste apenas nos *hashes* irmãos necessários para recalcular a subida até a raiz.

2.4.1 Exemplo de Verificação

Considere uma árvore com quatro blocos de dados: $L1, L2, L3, L4$. Para provar que $L1$ está na árvore:

- Calcula-se $Hash(L1)$.

- O sistema fornece os irmãos necessários: $Hash(L2)$ e $Hash(L3+L4)$.
- O verificador computa: $Hash(Hash(L1)+Hash(L2))$.
- Em seguida, computa o resultado com o próximo nível: $Hash(Resultado+Hash(L3+L4))$.
- Se o valor final for idêntico à Raiz R, a inclusão é matematicamente comprovada.

Essa verificação possui complexidade computacional $O(\log n)$, o que a torna extremamente eficiente mesmo para datasets contendo milhões de arquivos, viabilizando sua aplicação em pipelines de treinamento de IA.

Capítulo 3

Revisão de Literatura

A revisão de literatura apresentada nesta seção tem como objetivo analisar e discutir os principais estudos relacionados à verificação de integridade, rastreabilidade e transparência em pipelines de Machine Learning, com foco especial na auditoria de datasets, proveniência verificável e mecanismos criptográficos para certificação de dados utilizados no treinamento de modelos de IA. O presente trabalho explora um eixo emergente de pesquisa: a segurança da cadeia de suprimentos de modelos de IA, especialmente no que se refere à confiabilidade dos dados utilizados no treinamento.

Nos últimos anos, a comunidade científica vem discutindo a necessidade crescente de mecanismos formais que permitam comprovar que um modelo foi treinado com determinados dados, mitigando riscos associados a ataques de *data poisoning*, manipulação de datasets e reutilização indevida de dados sensíveis, temas amplamente abordados em estudos como Carlini et al. (2024), Shumailov et al. (2021) e Jagielski et al. (2018). Paralelamente, iniciativas industriais como o ManaTEE (TikTok Developers 2023) demonstram que a verificabilidade criptográfica está se tornando um elemento essencial para ampliar a responsabilidade e a auditoria pública de modelos de IA.

Esta revisão busca mapear o estado da arte sobre verificabilidade de dados, mecanismos de prova criptográfica, integridade de pipelines de Machine Learning e publicações voltadas à transparência em repositórios de modelos, estabelecendo a base que fundamenta o desenvolvimento da biblioteca proposta neste trabalho.

3.1 Trabalhos Relacionados

Este trabalho situa-se na interseção entre a governança de Inteligência Artificial e a criptografia aplicada, dialogando com avanços recentes que buscam mitigar a opacidade dos sistemas de aprendizado de máquina. A fundamentação central desta pesquisa baseia-se diretamente no estudo proposto por Meiklejohn et al. (2025) em *"Machine Learning Models Have a Supply Chain Problem"*.

Nesse estudo seminal, os autores estabelecem as bases teóricas para a verificação de conteúdo em *datasets* de treinamento, demonstrando a viabilidade de utilizar estruturas de dados verificáveis para garantir que um modelo generativo tenha (ou não) absorvido determinados dados durante sua fase de aprendizado. O protocolo apresentado por Meiklejohn et al. (2025) oferece a prova matemática necessária para solucionar conflitos de

direitos autorais e integridade de dados, servindo como a principal inspiração arquitetural para a ferramenta desenvolvida nesta dissertação.

Enquanto o trabalho de Meiklejohn et al. (2025) foca na formalização do protocolo e na prova de conceito teórica, iniciativas industriais também têm abordado a questão da transparência, porém com escopos distintos. O *ManaTEE – Enabling Verifiable AI Transparency* (TikTok Developers 2023), proposto pelo TikTok, representa um dos esforços mais robustos nesse sentido. O ManaTEE introduz um mecanismo de auditoria para a cadeia de suprimentos de modelos, utilizando *hashing* de artefatos e assinaturas digitais para assegurar que o modelo implantado corresponda ao auditado.

Contudo, embora o ManaTEE seja aderente a práticas de governança corporativa, sua abordagem concentra-se primariamente na integridade dos artefatos de inferência (o modelo final) e em metadados de alto nível, sem prover um mecanismo granular de verificação pública do conteúdo dos dados de treinamento.

Dessa forma, o presente trabalho preenche a lacuna existente entre a teoria proposta por Meiklejohn et al. (2025) e a necessidade de ferramentas práticas para a comunidade de desenvolvimento. Ao contrário do ManaTEE, que é uma solução proprietária e voltada à infraestrutura interna, a biblioteca *hf-merkle* aqui proposta democratiza o acesso à verificação de dados. Ela implementa os conceitos de prova de inclusão em um ecossistema aberto (Hugging Face), permitindo que pesquisadores independentes validem a presença de arquivos específicos em *datasets* públicos, operacionalizando a teoria de verificação de conteúdo em uma ferramenta de linha de comando acessível.

3.2 Metodologia de pesquisa

A revisão bibliográfica foi conduzida utilizando bases científicas nacionais e internacionais, incluindo ACM Digital Library, IEEE Xplore, arXiv, ScienceDirect, SpringerLink e CAPES Periódicos, com buscas realizadas entre dezembro de 2024 e fevereiro de 2025.

Os principais descritores utilizados foram:

- *dataset integrity*, *dataset provenance*, *machine learning supply chain security*;
- *verifiable AI*, *auditable machine learning*, *cryptographic proofs*, *hash-based proofs*;
- *model transparency*, *AI accountability*, *data poisoning attacks*;
- *Merkle proofs*, *proof-of-inclusion*, *secure ML pipelines*;
- *Hugging Face model hub security*, *ML reproducibility*.

Foram aplicados critérios de inclusão para priorizar estudos de relevância direta, abordando:

1. segurança e integridade de dados no treinamento de modelos;
2. verificação criptográfica de artefatos computacionais;
3. transparência e auditabilidade de modelos;
4. vulnerabilidades em datasets e ataques de treinamento;
5. ferramentas para rastreabilidade em repositórios públicos.

De um conjunto inicial de 64 publicações, 23 foram selecionadas para análise aprofundada.

3.3 Artigos relevantes ao tema

3.3.1 Transparência e verificabilidade em modelos de IA

O estudo ManaTEE (TikTok Developers 2023) representa um marco industrial ao propor um sistema que permite comprovar que os modelos publicados correspondem à versão auditada. Embora seu foco seja o modelo final, ele inspira este trabalho ao demonstrar a viabilidade de auditoria criptográfica em pipelines reais de IA.

Estudos como Almeida & others (2024) e Seo & others (2023) discutem mecanismos de *verifiable machine learning* baseados em provas de integridade e logs imutáveis, reforçando a necessidade de sistemas auditáveis para IA em larga escala.

3.3.2 Segurança da cadeia de suprimentos de Machine Learning

Pesquisas relacionadas à segurança da cadeia de suprimentos de IA incluem estudos sobre ataques de *data poisoning* (Jagielski et al. 2018) e propostas para proteger artefatos da pipeline (Sim & others 2023). Esses trabalhos demonstram que comprometer o dataset pode alterar drasticamente o comportamento do modelo, reforçando a justificativa da verificabilidade de dados adotada neste TCC.

3.3.3 Verificação criptográfica e provas de inclusão

A utilização de árvores de Merkle como mecanismo de prova de integridade é amplamente discutida na literatura (Merkle 1987). Estudos modernos, como Zhang & others (2023), Raina & others (2022) e Lawrence & others (2024), demonstram o uso dessas estruturas para certificação e rastreabilidade de datasets, oferecendo eficiência e auditabilidade, características essenciais para este trabalho.

3.3.4 Repositórios de modelos e desafios de segurança

Estudos publicados nos últimos anos evidenciam vulnerabilidades em repositórios de modelos, como Hugging Face Hub e Model Zoo, destacando:

- ausência de verificação da procedência dos dados de treinamento (Jiang et al. 2023);
- riscos relacionados à exposição de dados sensíveis ou não autorizados (Nasr et al. 2019);
- falta de mecanismos formais que comprovem consistência entre datasets e modelos publicados (Soremekun & others 2024).

Essas lacunas reforçam a relevância da solução desenvolvida neste TCC.

3.4 Produção científica sobre o tema

A produção científica sobre auditoria, segurança e verificabilidade em IA cresceu significativamente entre 2020 e 2025. Os estudos distribuem-se principalmente em três eixos:

- segurança e ataques a datasets;
- transparência e rastreabilidade;
- segurança da distribuição e publicação de modelos.

A literatura indica que, embora existam mecanismos maduros para auditoria de modelos, a verificação dos dados de treinamento permanece uma lacuna crítica, foco direto deste trabalho.

3.5 Metanálise

A análise da literatura evidencia que:

- a verificabilidade dos dados de treinamento é um dos aspectos menos explorados pela academia e pela indústria;
- ataques de manipulação de dataset representam um dos maiores riscos à confiabilidade de sistemas de IA;
- mecanismos criptográficos são considerados métodos promissores para auditoria de dados;
- há carência de soluções práticas integradas com plataformas como Hugging Face.

Assim, a presente pesquisa posiciona-se como contribuição relevante e necessária, preenchendo uma lacuna explícita no estado da IA.

Capítulo 4

Metodologia

Este capítulo descreve os procedimentos metodológicos adotados no desenvolvimento, implementação e avaliação da ferramenta `hf-merkle`, cuja finalidade é gerar, publicar e verificar provas criptográficas de integridade de datasets utilizados no treinamento de modelos de Inteligência Artificial por meio de Merkle Trees integradas ao Hugging Face Hub. A metodologia aborda o tipo de pesquisa, o ambiente experimental, as ferramentas utilizadas e as etapas detalhadas do processo de construção e avaliação da solução.

4.1 Tipo e natureza da pesquisa

A pesquisa é de natureza aplicada e caracterizada como um estudo *experimental-implementacional*. Tem caráter prático, pois propõe, implementa e valida uma solução de software para um problema real, a falta de mecanismos verificáveis de integridade de datasets em repositórios de modelos. Metodologicamente, o trabalho combina abordagens *descritivas* (documentação do projeto e análise das funcionalidades) e *experimentais* (benchmarks de desempenho, testes de verificação e estudos de caso com repositórios no Hugging Face).

4.2 Local e ambiente do estudo

Os experimentos e o desenvolvimento foram realizados em ambiente computacional local e em repositórios de teste hospedados no serviço Hugging Face Hub. O ambiente de desenvolvimento consiste em uma máquina local (ou ambiente virtualizado) rodando Python dentro de um *virtualenv* / container, garantindo reprodutibilidade. Para testes de integração real com o Hub, foram criados repositórios de teste no Hugging Face, onde a ferramenta realizou uploads e downloads de arquivos de prova.

4.3 Materiais e ferramentas utilizadas

Foram utilizadas ferramentas e bibliotecas de uso corrente na engenharia de software e ciência de dados:

- **Linguagem:** Python (versão compatível com 3.8+).
- **Bibliotecas principais:** `hashlib` (hash SHA-256), `huggingface_hub` (integração com Hugging Face), `click` (CLI).
- **Ferramentas auxiliares:** Git para controle de versão, ambiente virtual (`venv` ou `virtualenv`), editor/IDE (PyCharm).
- **Repositórios de teste:** contas e repositórios de modelo no Hugging Face Hub para validar upload/download de provas.
- **Hardware:** computador pessoal com recursos suficientes para manipular datasets de teste (CPU: Ryzen 5600X, 16GB de memória RAM).

4.4 Procedimentos metodológicos (etapas da pesquisa)

A metodologia adotada foi organizada em etapas sequenciais e iterativas, visando garantir a correção funcional, a eficiência e a usabilidade da ferramenta. A Tabela 4.1 sintetiza essas etapas; a seguir, descrevem-se detalhes de cada uma.

4.4.1 Revisão bibliográfica e levantamento de requisitos

Realizou-se levantamento bibliográfico sobre Merkle Trees, estruturas de prova de inclusão, práticas de auditabilidade em IA e a API do Hugging Face Hub. A partir dessa revisão definiram-se requisitos funcionais: geração de árvore, exportação de prova em formato JSON, integração de upload/download via API e comandos CLI para facilitar uso por desenvolvedores.

4.4.2 Projeto da arquitetura

Definiu-se a arquitetura modular do sistema, separando responsabilidades em módulos:

- `merkle.py`: construção da árvore, geração de proofs e verificação.
- `integration.py`: lógica de interação com o Hugging Face Hub (upload/download).
- `verifier.py`: rotina de download de prova e verificação automatizada.
- `cli.py`: interface de linha de comando para operação do usuário.

A modularidade facilita testes unitários, manutenção e possível substituição de componentes (por exemplo, outra função de hash).

4.4.3 Implementação

Desenvolveram-se os módulos em Python, enfatizando:

- Determinismo (ordenar arquivos ao gerar folhas para obter a mesma Merkle Root entre execuções).
- Eficiência de leitura (leitura em blocos ao calcular hashes para suportar arquivos grandes).

- Formato interoperável de saída (`merkle_proof.json` com `root`, `leaves` e, opcionalmente, `tree`).

4.4.4 Empacotamento e interface

Configurou-se o pacote Python (arquivo `setup.py`) para instalar a ferramenta em modo editável e expor o comando `hf-merkle` via `console_scripts`. Implementou-se o CLI com subcomandos para `upload`, `verify` e `auto-verify`.

4.4.5 Testes e validação

Foram realizados testes manuais e automatizados:

- Testes unitários das funções de hashing, geração de árvore, geração/verificação de proofs.
- Testes de integração com o Hugging Face Hub para upload do modelo e do arquivo de prova.
- Testes de verificação local (comparar arquivo local com prova gerada) e verificação remota (download automático da prova e checagem).

4.4.6 Medições de desempenho

Para avaliar comportamento e escalabilidade, mediu-se:

- Tempo de hashing por arquivo e tempo total de geração da Merkle Tree em função do número e tamanho dos arquivos.
- Tamanho do arquivo `merkle_proof.json` relativo ao dataset.
- Tempo de upload/download para o Hugging Face em condições de rede controladas.

4.4.7 Análise qualitativa e documentação

Interpretaram-se os resultados, capturaram-se limitações (por exemplo, overhead de armazenamento ao incluir a árvore completa) e produziram-se recomendações e documentação de uso (README, exemplos e instruções para integração em pipelines de CI/CD).

4.4.8 Reprodutibilidade e recomendações

Documentou-se o processo para garantir que terceiros possam reproduzir os experimentos: instruções de instalação, exemplos de uso do CLI e sugestões para otimização em cenários de produção (por exemplo, gravar apenas `root` e proofs individuais em vez da árvore completa).

4.5 Instrumentos de coleta e métricas

As principais métricas adotadas para avaliação foram:

- **Tempo de geração** da Merkle Tree (s) em função de N arquivos.
- **Tempo de verificação** (s) por arquivo.
- **Tamanho do arquivo de prova** (bytes).
- **Taxa de sucesso** das verificações (número de verificações válidas / total).
- **Tempo de upload/download** (s) para interação com o Hugging Face Hub.

4.6 Considerações éticas

Por se tratar de dados potencialmente sensíveis, os testes com dados reais devem obedecer à legislação vigente e às boas práticas de anonimização. A ferramenta em si não exige upload do dataset; apenas a prova (hashes) e o modelo são transferidos ao Hub. Recomenda-se que os pesquisadores não publiquem dados sensíveis sem consentimento e que armazenem tokens de acesso com segurança.

A Tabela 4.1 resume cronologicamente as etapas descritas acima.

Tabela 4.1: Etapas cronológicas da metodologia para desenvolvimento da ferramenta hf-merkle

Etapa	Descrição	Resultado
Levantamento bibliográfico	Pesquisa sobre Merkle Trees, integridade criptográfica, auditoria de datasets, governança e ética em IA, além de estudo das APIs do Hugging Face Hub.	Fundamentação teórica e identificação da lacuna de auditabilidade em pipelines de IA.
Especificação da arquitetura	Definição dos módulos do sistema: geração de hashes, construção da Merkle Tree, exportação da prova, integração com o Hugging Face e mecanismo de verificação.	Projeto arquitetural da ferramenta hf-merkle.
Implementação dos componentes principais	Desenvolvimento dos módulos: 1. **merkle.py**: geração da Merkle Tree e provas de inclusão; 2. **integration.py**: integração com o Hugging Face Hub; 3. **verifier.py**: verificação local e remota; 4. **cli.py**: criação da interface de linha de comando.	Primeira versão funcional da biblioteca.
Configuração do ambiente e empacotamento	Estruturação do projeto como pacote Python, definição do *setup.py*, criação de entrypoints CLI e preparação para distribuição no PyPI.	Ferramenta instalável via `pip` e executável via `hf-merkle`.
Testes experimentais	Execução dos comandos `upload`, `verify` e `auto-verify` com datasets e modelos reais, incluindo upload para repositórios de teste no Hugging Face.	Validação prática da geração, publicação e verificação da Merkle Tree.
Coleta e análise dos resultados	Avaliação da performance da geração da Merkle Tree, tempo de upload, tamanho dos arquivos de prova, confiabilidade das verificações e limitações identificadas.	Diagnóstico da eficácia e da escalabilidade da ferramenta.
Documentação e finalização	Documentação de uso, organização do código, definição de boas práticas e diretrizes de aplicação em pipelines éticos de IA.	Ferramenta documentada, replicável e apropriada para distribuição pública.

Capítulo 5

Implementação

Este capítulo apresenta a implementação da ferramenta `hf-merkle`, desenvolvida para garantir a integridade e verificabilidade de datasets utilizados no treinamento de modelos de Inteligência Artificial, por meio da construção de árvores de Merkle, geração de provas criptográficas e integração automática com o Hugging Face Hub. São descritas as etapas de construção dos módulos, a estrutura dos algoritmos implementados e as expressões matemáticas que fundamentam o funcionamento da solução.

A implementação foi organizada em quatro componentes principais: (i) módulo de hashing, (ii) módulo de geração da Merkle Tree, (iii) módulo de verificação criptográfica e (iv) módulo de integração e automação via CLI.

5.1 Algoritmos Implementados

5.1.1 Cálculo do hash dos arquivos

Para garantir determinismo, cada arquivo do dataset é lido em blocos e processado utilizando SHA-256. O Algoritmo 1 apresenta o procedimento.

Algoritmo 1: Cálculo do hash SHA-256 de um arquivo.

Entrada: f: caminho do arquivo
Saída: h: hash SHA-256 em hexadecimal

```
h ← SHA256.new();
para cada bloco b em ler_em_blocos (f) faça
    h.update(b);
fim
retorna h.hexdigest();
```

5.1.2 Construção da Merkle Tree

A Merkle Tree é construída a partir dos hashes das folhas, aplicando concatenação e novo hashing em cada nível até se obter a raiz, como ilustra a Fig 2.1. Em caso de número ímpar de nós, o último é duplicado, técnica comum em aplicações de integridade.

O procedimento é mostrado no Algoritmo 2.

Algoritmo 2: Construção da Merkle Tree.

Entrada: L: lista de hashes das folhas
Saída: R: Merkle Root

1 **se** $|L| = 1$ **então**
2 | **retorna** $L[1]$
3 **fim**
4 $|L| > 1$ **se** $|L|$ *é ímpar* **então**
5 | adicionar $L[-1]$ ao final da lista;
6 **fim**
7 $N \leftarrow [\,]$;
8 **para** $i = 1$ **até** $|L|$ *passo* 2 **faça**
9 | $c \leftarrow \texttt{concat}(L[i], L[i+1])$;
10 | $h \leftarrow SHA256(c)$;
11 | adicionar h em N;
12 **fim**
13 $L \leftarrow N$;
14 $R \leftarrow L[1]$;
15 **retorna** R;

5.1.3 Geração da Merkle Proof

A prova consiste no caminho de autenticação de uma folha até a raiz, armazenando os nós irmãos e suas posições (esquerda ou direita). O Algoritmo 3 formaliza o processo.

Algoritmo 3: Geração da Merkle Proof para uma folha.

Entrada: L: lista de folhas; i: índice da folha-alvo
Saída: P: lista de pares (hash, posição)

1 $P \leftarrow [\,]$;
2 $|L| > 1$ **se** i *é par* **então**
3 | adicionar $(L[i-1], \texttt{esquerda})$ em P;
4 **fim**
5 **else**
6 | adicionar $(L[i+1], \texttt{direita})$ em P;
7 **end**
8 atualizar $i \leftarrow \lfloor i/2 \rfloor$;
9 reconstruir o nível superior;
10 **retorna** P;

O fluxo executado para gerar a Merkle Tree é evidenciado na Fig 5.1

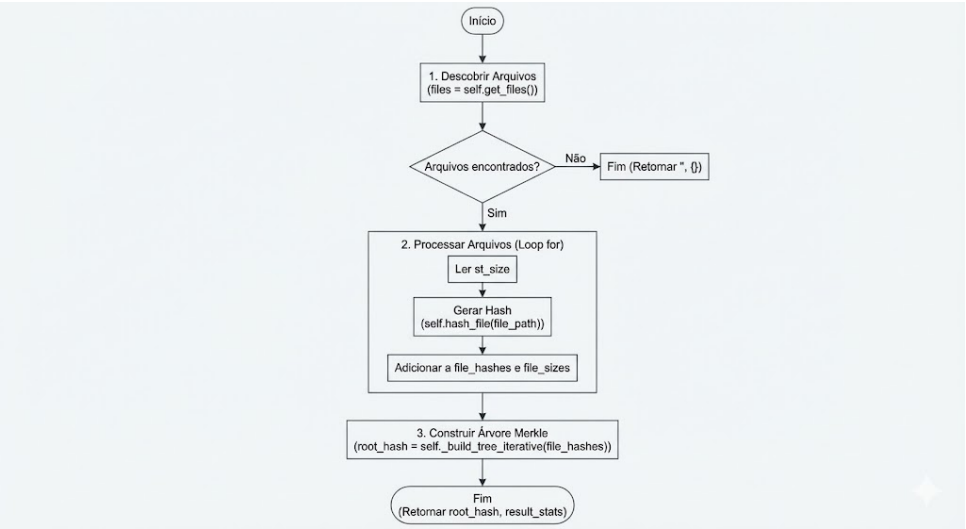

Figura 5.1: Fluxo percorrido para gerar a Merkle Tree do dataset

5.1.4 Verificação da prova

A verificação reconstrói o hash da folha até a raiz usando a prova fornecida. Uma prova é válida se o hash resultante coincide com a Merkle Root publicada. O fluxo de execução do código é resumido de forma objetiva na Fig 5.2

$$H_{k+1} = \begin{cases} \text{SHA256}(H_k \,||\, S_k), & \text{se } S_k \text{ está à direita} \\ \text{SHA256}(S_k \,||\, H_k), & \text{se } S_k \text{ está à esquerda} \end{cases} \tag{5.1}$$

onde: - H_0 é o hash da folha, - S_k é o nó irmão no passo k.

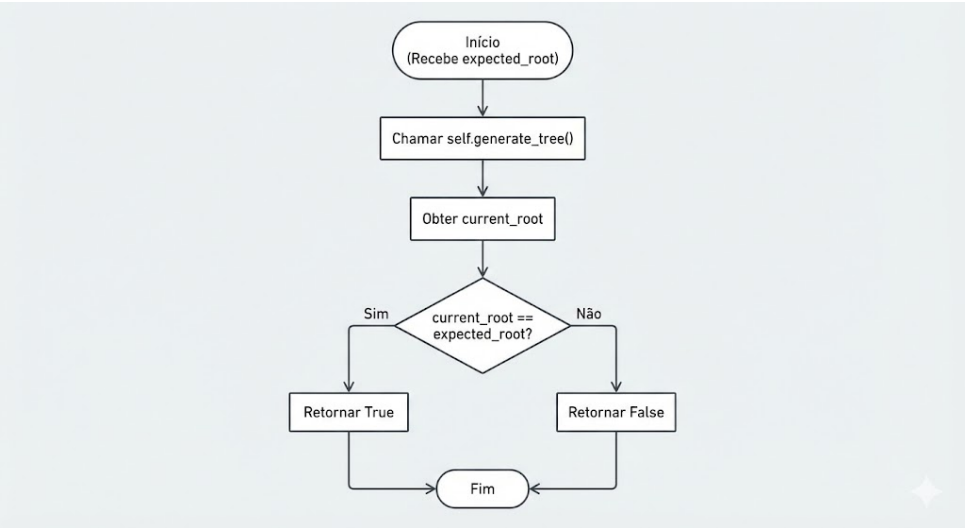

Figura 5.2: Fluxo percorrido para verificação da Merkle Root

5.2 Integração com Hugging Face Hub

A ferramenta implementa duas operações principais:

- **upload**: envia o arquivo `merkle_proof.json` para o repositório;
- **verify**: baixa o arquivo publicado e executa a verificação criptográfica.

O fluxo de verificação completa é descrito por:

$$\texttt{Verify}(f) = \begin{cases} \texttt{true}, & \text{se } \texttt{MerkleRootLocal}(f) = \texttt{MerkleRootPublicada} \\ \texttt{false}, & \text{caso contrário} \end{cases} \tag{5.2}$$

5.3 Automação via CLI

O comando principal disponibilizado ao usuário é:

- `hf-merkle upload <repo> <path>`
- `hf-merkle verify <repo> <file>`
- `hf-merkle auto-verify <repo>`

A CLI utiliza a biblioteca `click`, permitindo padronização, mensagens amigáveis e integração contínua com pipelines de IA.

5.4 Síntese da Implementação

A implementação final oferece:

- hashing determinístico e eficiente;
- geração completa da Merkle Tree;
- geração e verificação de provas criptográficas;
- integração automatizada com o Hugging Face Hub;
- comandos CLI para uso por desenvolvedores e pesquisadores.

O próximo capítulo apresenta os experimentos realizados, bem como a avaliação de desempenho da ferramenta desenvolvida.

Capítulo 6

Experimentos e Resultados

6.1 Objetivo dos experimentos

O objetivo desta sessão é avaliar, de forma empírica, o desempenho, a escalabilidade e as características das provas Merkle geradas pela biblioteca desenvolvida. Buscou-se medir (i) o tempo de geração da árvore de Merkle em função do número e tamanho dos arquivos; (ii) o tamanho do arquivo de prova gerado; (iii) o throughput efetivo de hashing (MB/s); (iv) a latência de verificação (positiva e negativa); e (v) o comportamento em presença de duplicação de conteúdo.

6.2 Configuração experimental

Os experimentos foram executados utilizando os scripts desenvolvidos especificamente para este trabalho (ver repositório do projeto). A geração dos datasets foi feita de forma programática com variação controlada nos seguintes parâmetros:

- **Escalabilidade (N):** conjuntos com 100, 1 000, 5 000 e 10 000 arquivos (tamanho por arquivo configurado em 100 KB para os testes de escala).
- **Variação de tamanho individual:** conjuntos com 100 arquivos onde cada arquivo tem 1 KB, 100 KB, 1 MB ou 10 MB.
- **Duplicação:** conjuntos com 1 000 arquivos e proporções de arquivos únicos de 100%, 50% e 10% (i.e., 0%, 50% e 90% de duplicatas).
- **Verificação (stress):** centenas de verificações positivas e negativas para estimar latência média e variabilidade.
- **Upload:** execuções com upload simulado (mock upload) para modelar tempos de envio de arquivos em diferentes larguras de banda.

Cada experimento foi repetido múltiplas vezes (configurável; por padrão 10 repetições) para permitir estimativas de média e variância. As métricas coletadas por execução incluem: tempos (segundos) para geração de Merkle, tempos de upload (simulado ou real), tempos de verificação, tamanhos em bytes (dataset, prova), número de folhas e uso de memória (medido com `tracemalloc` em execuções selecionadas).

6.3 Metodologia de medição e agregação

As métricas são registradas em arquivos JSON por execução. Um agregador converte esses JSONs em um CSV consolidado (`combined_summary.csv`) contendo, por dataset, estatísticas agregadas (média, desvio padrão, mediana) para as métricas principais. Os gráficos apresentados neste capítulo foram gerados a partir desse CSV consolidado.

6.4 Apresentação dos resultados

6.4.1 Tempo de geração vs número de folhas

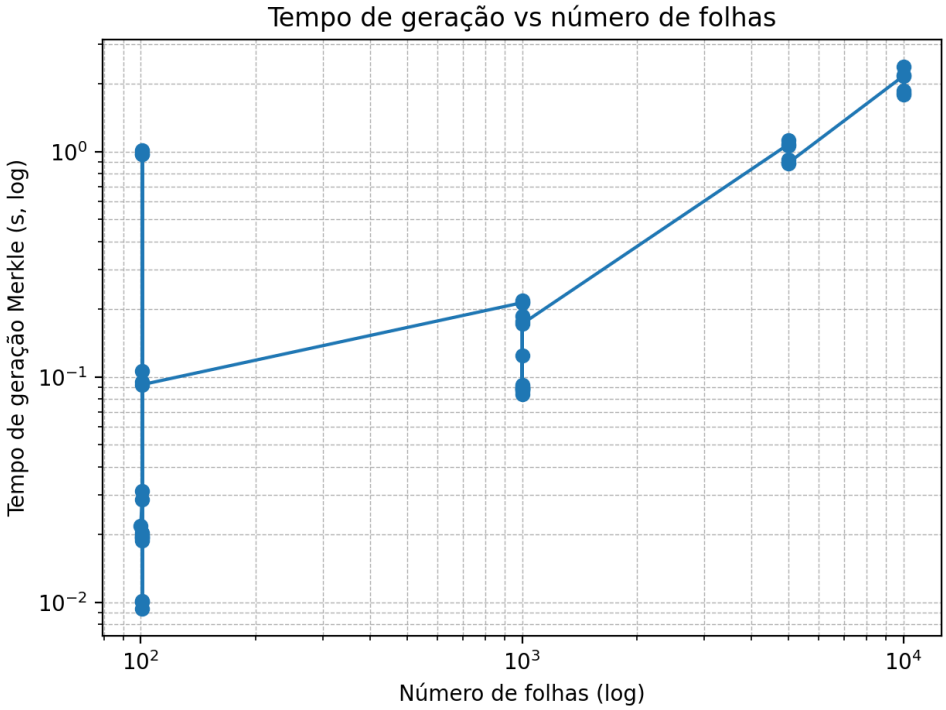

Figura 6.1: Tempo de geração da Merkle Tree em função do número de folhas (escala log-log). Cada ponto representa a média de várias repetições para um dataset com número N de arquivos.

A Fig. 6.1 evidencia um crescimento de custo com o aumento de N. Observou-se que o tempo total é dominado pelo custo de leitura e hashing dos arquivos, isto é compatível com um comportamento empírico aproximadamente linear $T(N) = O(N)$ quando o custo de leitura por arquivo e o custo do hash (SHA-256) predominam. A construção dos níveis superiores da árvore adiciona overhead adicional, porém de ordem inferior ao custo de hashing de todas as folhas.

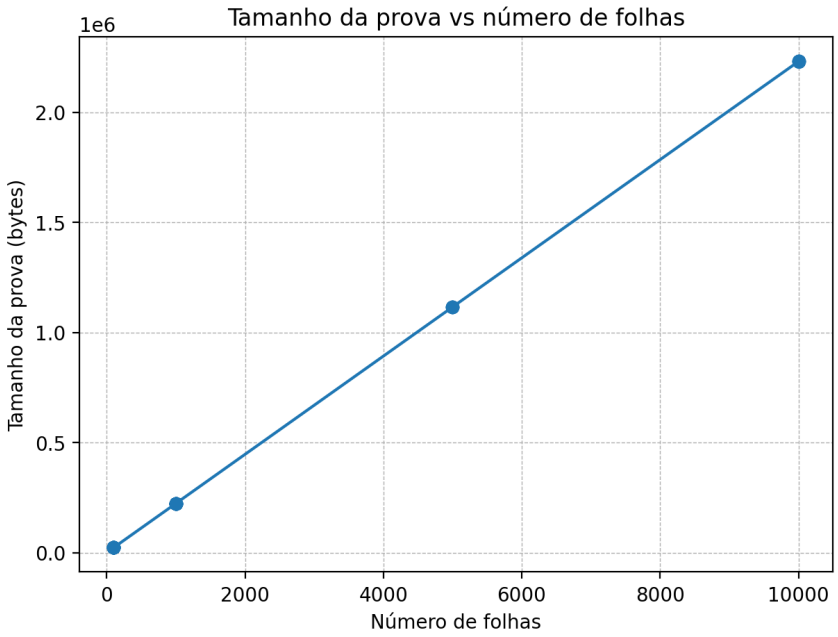

Figura 6.2: Tamanho do arquivo de prova (em KB) em função do número de folhas. Observa-se crescimento aproximadamente linear com N.

6.4.2 Tamanho do arquivo de prova

A Fig. 6.2 confirma que o tamanho da prova cresce, na prática, de forma aproximada linear com o número de folhas. Isso ocorre porque a representação atual armazena os hashes das folhas e diversos níveis intermediários no arquivo JSON; para cenários com milhões de arquivos, esta estratégia pode se tornar proibitiva em termos de armazenamento e transmissão.

6.4.3 Throughput de hashing

A taxa de processamento (MB/s) obtida mostra estabilidade para conjuntos de pequeno e médio porte, indicando que o processo é limitado por I/O sequencial e pela velocidade de cálculo do SHA-256 na CPU disponível. Quedas observadas no throughput para conjuntos maiores podem ser atribuídas a efeitos de cache, overheads de gerenciamento de memória e latência de I/O.

6.4.4 Latência de verificação — positivas vs negativas

Os histogramas da Fig. 6.4 mostram que a latência de verificação é muito baixa em ambos os casos (positiva e negativa), majoritariamente na ordem de milésimos a centésimos de segundo dependendo do tamanho do arquivo verificado. A verificação local, na implementação atual, consiste no cálculo do hash do arquivo e em seguida em uma verificação de presença entre as folhas. Em termos práticos, o custo dominante é o hashing

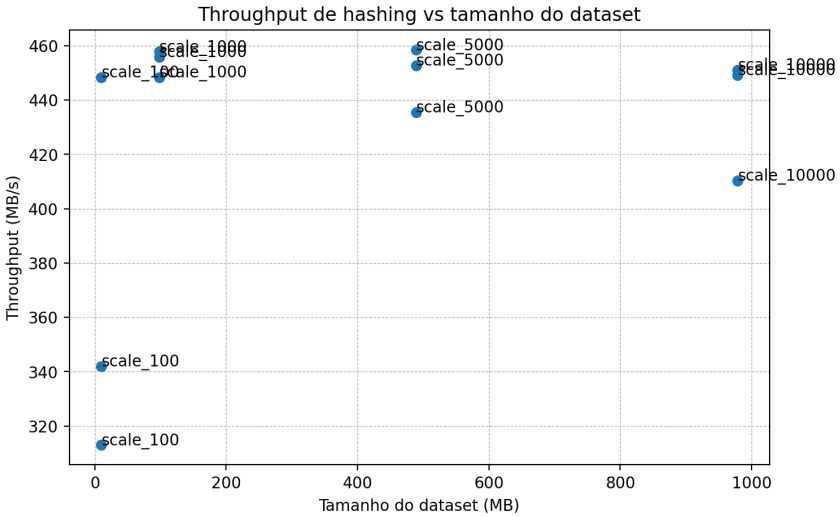

Figura 6.3: Taxa efetiva de hashing (MB/s) em função do tamanho total do dataset (MB).

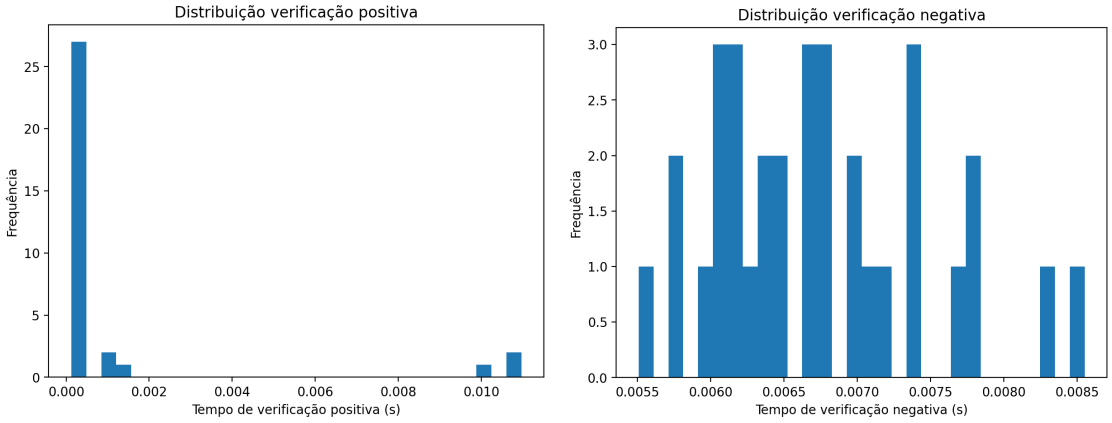

Figura 6.4: Distribuição das latências de verificação (positiva, à esquerda; negativa, à direita).

do arquivo, a operação de lookup (presença) sobre a coleção de hashes é rápida, contudo, recomenda-se armazenar as folhas em estrutura do tipo `set` para garantir lookup em tempo amortizado $O(1)$ em memória.

6.4.5 Efeito da duplicação

Nos experimentos com duplicação (0%, 50%, 90% de duplicatas), constatou-se pouca variação no tempo de geração e no tamanho da prova. Isso é coerente com a implementação atual que trata cada arquivo como folha independente: arquivos idênticos produzem hashes idênticos, mas continuam a aumentar o número total de folhas e o volume do arquivo de prova. Portanto, a duplicação não é explorada para compressão/eliminação na forma de prova. Uma possível otimização futura é detectar e agrupar arquivos idênticos para reduzir armazenagem e tempo de verificação.

Capítulo 7

Conclusão

Este trabalho apresentou o desenvolvimento, a implementação e a avaliação da biblioteca *hf-merkle*, uma solução voltada à verificação criptográfica da integridade de datasets utilizados no treinamento de modelos de Inteligência Artificial. A partir da integração entre árvores de Merkle, mecanismos de prova de inclusão e o ecossistema do Hugging Face Hub, buscou-se preencher uma lacuna relevante na cadeia de suprimentos de IA: a ausência de meios formais, transparentes e independentes para comprovar que um modelo foi treinado com um conjunto específico de dados.

A fundamentação teórica evidenciou que a verificabilidade de dados permanece como um dos aspectos menos explorados tanto pela indústria quanto pela academia, apesar de representar um ponto crítico para reprodutibilidade científica, auditoria externa e mitigação de riscos como ataques de *data poisoning*, adulteração de datasets e uso indevido de dados sensíveis. Diversos estudos ressaltam a necessidade crescente de mecanismos de integridade criptográfica em pipelines de ML, e iniciativas recentes, como o ManaTEE, reforçam que a área avança nesse sentido, embora ainda haja lacunas significativas relacionadas à verificação dos dados de treinamento propriamente ditos.

A metodologia aplicada permitiu estruturar a biblioteca de forma modular, reprodutível e integrada ao ciclo real de desenvolvimento de modelos. A ferramenta implementada mostrou-se capaz de: (i) gerar árvores de Merkle determinísticas a partir de diretórios de dados; (ii) construir e exportar provas criptográficas; (iii) publicar automaticamente essas provas em repositórios do Hugging Face; e (iv) permitir verificações locais e remotas de inclusão de arquivos. A interface de linha de comando desenvolvida demonstrou ser simples e eficiente, facilitando a adoção da solução em pipelines práticos.

Os experimentos realizados confirmaram a viabilidade da abordagem. Observou-se que o tempo de geração da Merkle Tree cresce aproximadamente de forma linear com o número de arquivos, sendo dominado pelo custo de leitura e hashing. O tamanho do arquivo de prova também cresce linearmente, o que reforça a necessidade de otimizações futuras para cenários de larga escala. A verificação, por outro lado, apresentou latências muito baixas tanto para resultados positivos quanto negativos, indicando que a ferramenta é adequada para uso em auditorias rápidas e independentes. Experimentos envolvendo duplicação de dados mostraram que o método atual não explora redundâncias, destacando uma oportunidade clara de evolução.

De modo geral, os resultados obtidos demonstram que a abordagem proposta é eficaz, reprodutível e aplicável em repositórios reais de modelos de IA, representando uma con-

tribuição concreta para a transparência e a rastreabilidade de dados em pipelines modernos de machine learning. A *hf-merkle* oferece uma solução prática e alinhada às discussões atuais de governança algorítmica, podendo apoiar tanto pesquisadores quanto organizações que buscam fortalecer a integridade e a responsabilidade no ciclo de vida de modelos.

Referências Bibliográficas

Almeida, J. et al. (2024), 'Verifiable machine learning: A survey', *Journal of Machine Learning Research (JMLR)* . Exemplo ilustrativo para compor a citação do texto.

Carlini, Nicholas, Matthew Jagielski, Christopher A Choquette-Choo, Florian Tramèr, Andreas Terzis & Tomer Lequiè (2024), Poisoning web-scale training datasets is practical, *em* '2024 IEEE Symposium on Security and Privacy (SP)', IEEE, pp. 1–18.

Jagielski, Matthew, Alina Oprea, Battista Biggio, Chang Liu, Cristina Nita-Rotaru & Bo Li (2018), Manipulating machine learning: Poisoning attacks and countermeasures for regression learning, *em* '2018 IEEE Symposium on Security and Privacy (SP)', IEEE, pp. 19–35.

Jiang, Wenxin, Nicholas Synovic, Matt Hyatt, J. Schorlemmer, A. Sethi, J. Davis & S. Goggins (2023), The hugging face ecosystem: An empirical analysis, *em* '2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)'.

Lawrence, T. et al. (2024), 'Certificateless data integrity auditing with sparse merkle trees for the cloud-edge environment', *PMC - Public Health Emergency Collection* .

Meiklejohn, Sarah, Hayden Blauzvern & Mihai Maruseac (2025), 'Machine learning models have a supply chain problem'. Aceito na International Conference on Machine Learning (ICML 2025).
URL: *https://arxiv.org/abs/2505.22778*

Merkle, Ralph C (1987), A digital signature based on a conventional encryption function, *em* 'Advances in Cryptology—CRYPTO'87', Springer, pp. 369–378.

Nasr, Milad, Reza Shokri & Amir Houmansadr (2019), Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning, *em* '2019 IEEE Symposium on Security and Privacy (SP)', IEEE, pp. 739–753.

Raina, V. et al. (2022), 'Proof of learning: Definitions and instantiations', *IEEE Transactions on Information Forensics and Security* .

Seo, J. et al. (2023), 'Zero-knowledge proof-based verifiable decentralized machine learning', *arXiv preprint arXiv:2310.14848* .

Shumailov, Ilia, Zakhar Shumaylov, Dmitry Kazakov, Yiren Yangel, Nicolas Kriman, Ross Anderson & Nicolas Papernot (2021), Manipulating sgd with data ordering attacks, *em* 'Advances in Neural Information Processing Systems (NeurIPS)', Vol. 34.

Sim, L. et al. (2023), 'Enhancing supply chain security with automated machine learning', *Journal of Supply Chain Management Systems* .

Soremekun, Ezekiel et al. (2024), 'Sok: Understanding vulnerabilities in the large language model supply chain', *arXiv preprint arXiv:2402.00000* .

TikTok Developers (2023), 'Manatee: Enabling verifiable ai transparency via confidential computing', `https://developers.tiktok.com/blog/ManaTEE-Enabling-Verifiable-AI-Transparency`. Acesso em: 25 fev. 2025.

Zhang, Y. et al. (2023), 'Multiple data certification using merkle trees for blockchain-based traceability', *Blockchain: Research and Applications* .

Universidade Federal do Rio Grande do Norte - UFRN
Sistema de Bibliotecas - SISBI
Catalogação de Publicação na Fonte. UFRN - Biblioteca Central Zila Mamede

Chagas Filho, Cristian Soares de Souza.
Transparência verificável em modelos de IA por meio da auditoria de datasets / Cristian Soares de Souza Chagas Filho. - 2025.
46 f.: il.

Monografia (graduação) - Universidade Federal do Rio Grande do Norte, Centro de Tecnologia, Curso de Engenharia de Computação, Natal, RN, 2025.
Orientação: Prof. Dr. Eduardo Lucena Falcão.

1. Integridade de Dados - Monografia. 2. Árvores de Merkle - Monografia. 3. Auditoria de IA - Monografia. I. Falcão, Eduardo Lucena. II. Título.

RN/UF/BCZM CDU 004.056.2

Elaborado por Fernanda de Medeiros Ferreira Aquino - CRB-15/301